



ENME 441

04 – Communication Buses

SPI · SoftSPI · I2C · UART · JTAG

Communication Options on the ESP32-C3



SPI

Fast, clocked, 4 wires

- Signals and modes
- machine.SPI
- Bit-banging by hand
- SoftSPI vs hardware



I2C

2 wires, many devices

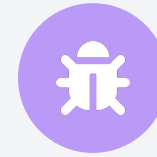
- Addresses and ACKs
- Pull-up resistors
- scan() and readfrom()
- 100 kHz vs 400 kHz



UART

No shared clock, just a baud rate

- Point-to-point only
- Asynchronous (no shared clock)
- UART0 is your REPL



JTAG

Debug port

- What JTAG actually does
- The built-in USB unit
- JTAG owns your COM port
- Pin collisions

Key Differences

1

Synchronous (clocked) vs. asynchronous

Synchronous buses (SPI and I2C) use a separate clock wire to tell the receiver when to read the data line. Asynchronous buses (UART) do not share a clock, but instead both ends agree on a set data rate, with extra bits used to frame each data block.

2

Can you talk to more than one device?

SPI gives every device its own chip-select wire, so wiring grows with device count.
I2C puts a 7-bit address in the data itself, so all devices use just 2 wires.
UART has no addressing at all — one link, one device.

3

How fast, and how far?

SPI runs at MHz clock speed but only across a circuit board.
I2C sits at 100 or 400 kHz over a few centimetres.
UART is slow — 9600 to 115200 bits per second — but survives a long cable (good for board-to-board communication).

ESP32-C3 Bus Pins



1 × general-purpose SPI

The C3 has 3 SPI controllers, but SPI0 and SPI1 are wired to the internal flash and are off-limits. SPI2 is available for your use, but note that MicroPython calls it SPI(1).



1 × I2C controller

One hardware I2C, addressed as I2C(0), with SoftI2C providing software busses (as many as you have spare pins).



2 × UART

UART0 is used for the serial console behind your REPL. UART1 is available for your use.



1 × USB Serial/JTAG controller

Hard-wired to GPIO18 and GPIO19 — which the SuperMini does not break out, because they go straight to the USB-C connector.



The GPIO matrix

Between the bus controllers and the physical pins sits a routing matrix: almost any peripheral signal can be sent to almost any GPIO. So SDA, MOSI and TXD on the pinout above are defaults, not wiring rules. You name the pins in the constructor and the matrix does the rest.

01

SPI

Serial Peripheral Interface (SPI)
One wire per device



Serial Peripheral Interface (SPI)

SCK

Clock. One bit per edge.

MOSI

Controller Out, Peripheral In

MISO

Controller In, Peripheral Out.

CS / SS

Chip select, active LOW. One wire per device.

1

Full duplex communication

Each clock edge simultaneously pushes one bit on both the MOSI and MISO lines (communication goes both directions at once)

2

Nothing acknowledges anything

There is no address, no ACK bit, no error detection. If the wire falls off, your code carries on writing happily. This is the price SPI pays for being simple and fast.

3

Megahertz speeds

An LED driver is happy at 1 MHz; SD cards and colour displays run at 10–40 MHz. Those rapid edges do not survive a long cable, so SPI stays on the PCB or a short ribbon.

4

4 modes, set by polarity and phase

polarity picks the clock's idle level; phase picks whether the peripheral samples on the leading or trailing edge. The four combinations are SPI modes 0–3, and the device's datasheet tells you which.

Hardware SPI

Name the pins, set the mode, and drive chip select yourself.

```
from machine import Pin, SPI

spi = SPI(1,                # the C3's one usable controller
          baudrate=1000000, # 1 MHz
          polarity=0,       # \
          phase=0,          # / together: SPI mode 0
          sck=Pin(5),        # clock, driven by the controller
          mosi=Pin(6),       # data sent out to peripheral
          miso=Pin(7))       # response received by peripheral

cs = Pin(0, Pin.OUT, value=1) # chip select, idle HIGH

cs.value(0)                  # select the device...
spi.write(b'\x3F')           # ...send it a byte...
cs.value(1)                  # ...and release it
```

- **SPI(1) is the only choice**

Other SPI controllers are used to communicate with the flash chip.

- **The pins are arguments, not constants**

sck and mosi can be any free GPIO.
Leave miso out entirely for a write-only device.

- **baudrate is a request, not a promise**

The driver picks the closest divider it can reach from the peripheral clock. Read spi back with `print(spi)` to see what you actually got.

- **Chip select is yours to drive**

The SPI object never touches CS — it is plain GPIO that you pull LOW before a transfer and HIGH again after. Every device on the bus gets its own CS pin, and all of them stay HIGH except the one you are addressing.

Software SPI

THE SAME THING using `machine.SoftSPI`

```
from machine import Pin, SoftSPI
spi = SoftSPI(baudrate=100000,
              polarity=0, phase=0,
              sck=Pin(5), mosi=Pin(6),
              miso=Pin(7))
spi.write(b'\xA5')
```

1

No dedicated SPI hardware is involved — the CPU toggles the pins

Every clock edge is a machine instruction rather than a hardware timer tick, so the waveform is whatever the interpreter had time to produce. Garbage collection, an incoming WiFi packet or a timer callback will all stretch a clock pulse.

2

Slower by a factor of ten to a hundred

Hardware SPI reaches megahertz; SoftSPI on the C3 tops out in the low hundreds of kHz. For an LED driver updating a few times a second that is invisible. For an SD card or a colour display it is not.

3

Only use on the C3 when an extra bus is needed

On older ESP32s hardware SPI was tied to fixed pins, so SoftSPI was how you reached the others. The C3's GPIO matrix solves this, so only use SoftSPI when you need an additional SPI bus beyond the one hardware controller you have (e.g. when shared hardware lines cause conflicts with other devices).

02

I2C

Two wires, many devices



I2C — Inter-Integrated Circuit

Two wires shared by every device on the bus, with an address at the front of every conversation.

SDA

Serial data — bidirectional, one byte at a time.

SCL

Serial clock, driven by the controller.

4.7 kΩ ↑

Pull-ups to 3.3 V. Both lines need them.

1

Every device has a 7-bit address

The controller puts the address on the bus first; the device that recognises it answers and the rest stay quiet. Seven bits provides 112 usable addresses (128 minus 16 addresses reserved by the system).

2

Open-drain on device end: devices can only pull LOW

Nothing on an I2C bus ever drives a line HIGH — the pull-up resistors do that, and a device signals a 0 by pulling the line down. That is what lets many devices share one wire without ever fighting each other — and why a bus with no pull-ups reads as permanently busy.

3

Every byte is acknowledged

The receiver pulls SDA low for a ninth bit to say it heard you. That is how `i2c.scan()` works: it addresses all 128 possibilities and reports the ones that ACK. No other bus on the ESP32 will tell you whether the device is even plugged in.

4

100 kHz standard, 400 kHz fast — and half duplex

Far slower than SPI — one direction at a time on one data wire. Fine for reading a sensor every second; wrong for large data streams.

I2C – Reading and Writing Bytes

Find out what is on the bus, then read a register from it.

```
from machine import Pin, I2C

i2c = I2C(0, scl=Pin(9), sda=Pin(8), freq=400000)

print(i2c.scan())           # -> [60, 118]   (0x3C, 0x76)

# Read 2 bytes from register 0xF7 of the device at 0x76
DEV = 0x76    # I2C device address
REG = 0xF7    # register to read from device
NUM_BYTES    # number of bytes to read from register
data = i2c.readfrom_mem(DEV, REG, NUM_BYTES)
print(data)   # -> b'\x51\x40'

# Write one byte to a register on the same device
i2c.writeto_mem(DEV, 0xF4, b'\x27')
```

- **scan() prints decimal, datasheets print hex**
60 is 0x3C. Convert before you go looking in the datasheet, or you will conclude the wrong device answered.
- **An empty list means wiring, not code**
No pull-ups, swapped SDA and SCL, or no power to the breakout. Most breakout boards include the pull-ups already — check before adding your own.
- **readfrom_mem does two transactions**
It writes the register number, then reads back — the pattern nearly every I2C sensor expects, wrapped into one call.
- **SDA and SCL default to GPIO8 and GPIO9**
Both are strapping pins, and GPIO8 also drives the on-board LED. I2C is safe there because the pull-ups hold both HIGH at reset.

03

UART

No clock, just a baud rate



UART — Universal Asynchronous Receiver/Transmitter



1

2-wire point-to-point communication (one device per UART)

No addresses, no chip selects, no sharing. Just cross the 2 wires (TX to RX, RX to TX) using a shared ground, and that is the whole bus. Full duplex, since TX and RX are separate wires.

2

Asynchronous – both ends guess the timing

There is no shared clock to follow, so each end runs its own and the two must be configured to the same baud rate in advance. Get it wrong and you will see plausible-looking garbage rather than an error.

3

8N1: ten bits on the wire for every eight you send

1 start bit marks the beginning, then 8 data bits, and 1 stop bit. No parity check (extra bit in the data packet to indicate if sum of bits in packet is odd or even). 115200 baud carries about 11.5 kB of data per second.

4

Slow, but it travels

A few metres on plain wire, and hundreds with RS-485 or RS-232 level shifting. That robustness is why GPS receivers, LoRa radios, and board-to-board links still use it.

UART in MicroPython

Configure UART1, write bytes, read whatever came back.

```
from machine import Pin, UART
import time

uart = UART(1, baudrate=9600, tx=Pin(0), rx=Pin(1))

uart.write(b'AT\r\n')          # bytes, not str
time.sleep_ms(200)

if uart.any():                 # bytes waiting in the buffer?
    line = uart.readline()     # None if nothing arrives in time
    print(line.decode().strip())

# Non-blocking read of whatever is there
data = uart.read()             # None, or a bytes object
```

- **Use UART(1) — never UART(0)**

UART0 is the serial console carrying your REPL. Reconfigure it and Thonny loses the board mid-statement.

- **Everything is bytes**

write() takes bytes, read() returns bytes. Call .decode() to get text, .encode() to send it — the b" prefix is not decoration.

- **any() before read()**

read() blocks until the timeout expires. Checking any() first keeps a polling loop responsive; readline() still returns None on a timeout, so guard against it.

- **Baud rate is the first thing to check**

Consistent garbage on screen almost always means the two ends disagree. GPS modules default to 9600, most everything else to 115200.

04

JTAG

Not a bus — a debug port



JTAG — Joint Test Action Group

The other four buses connect the chip to peripherals. This one connects a debugger to the CPU itself.

TCK

Test clock, driven by the debug probe.

TMS

Test mode select — walks the state machine.

TDI

Test data in, towards the chip.

TDO

Test data out, back to the probe.

1

It reaches inside the processor, not out to a sensor

Halt the CPU mid-instruction, set breakpoints, single-step, read and write registers and memory, and program the flash. It was standardised (IEEE 1149.1) for testing solder joints on finished boards, and the debug access came later.

2

The C3 has the debug probe built in

A USB Serial/JTAG controller sits on the chip at GPIO18 and GPIO19, wired straight to the USB-C connector. Every other chip in this class needs a separate \$50 probe; the C3 needs the cable already in your hand.

3

That one cable carries two things at once

A CDC serial port — which is your REPL — and a JTAG debug channel, multiplexed over the same USB connection. This is why the port shows up as "USB JTAG/serial debug unit" in Device Manager, and why picking it in Thonny works.

4

You will not use the JTAG half in MicroPython

MicroPython gives you the REPL and `print()`; source-level debugging over JTAG is an ESP-IDF and C workflow, driven by OpenOCD and gdb. Worth knowing it is there for when you drop below MicroPython.

Thirteen Pins, and Some Are Already Taken

The buses overlap each other and the board's own functions. This is the map to check before you wire anything.

GPIO	Default bus role	What else wants it	Safe to reuse?
0 · 1	—	ADC1 channels 0 and 1	Yes — the freest pins on the board
2	—	Strapping pin, read at reset	Only if nothing pulls it LOW at boot
3	—	ADC1 channel 3	Yes
4 · 5 · 6 · 7	SPI: SCK, MISO, MOSI, SS	External JTAG: MTMS, MTDI, MTCK, MTDO	Yes — external JTAG is off by default
8	I2C: SDA	On-board LED, strapping pin	Yes for I2C — pull-ups hold it HIGH at reset
9	I2C: SCL	BOOT button, strapping pin	Yes for I2C — same reason
10	—	Selects external vs USB JTAG	Yes in practice
18 · 19	USB D- / D+	The built-in USB Serial/JTAG unit	Not broken out — you cannot reach them
20 · 21	UART0: RXD, TXD	The REPL and the serial console	Only if you give up the REPL over serial

The Espressif datasheet assigns GPIO20 to U0RXD and GPIO21 to U0TXD. Some widely-circulated SuperMini pinout diagrams print those two labels the other way round — if a UART link stays silent, swapping the pair is the first thing to try.

Side by Side

The whole lecture on one slide.

	SPI (hardware)	SPI (software)	I2C	UART	JTAG
Wires (plus GND)	4 + 1 CS each	4 + 1 CS each	2, shared	2, crossed over	4
Clock line	Yes — SCK, from the controller	Yes, toggled by the CPU	Yes — SCL, from the controller	None. Both ends agree on a baud rate	Yes — TCK, from the probe
Devices per bus	Many — one CS pin each	Many — one CS pin each	Up to 112, by address	Exactly one	A daisy-chain of many
How a device is picked	A dedicated chip-select wire	A dedicated chip-select wire	7-bit address, and it ACKs	It is the only one there	Position in the scan chain
Duplex	Full — always both ways	Full	Half — one way at a time	Full — separate TX and RX	Full
Typical speed	1–40 MHz	~100–300 kHz	100 or 400 kHz	9600–115200 baud (nominally 4800 – 57600 Hz)	MHz, limited by cable
On the ESP32-C3	One controller: SPI(1)	As many as you have pins	One: I2C(0), plus SoftI2C	Two: UART0 is the REPL, UART1 is yours	Built into the USB-C port
Reach for it when	Displays, SD cards, LED drivers	You need a second SPI bus	Sensors, RTCs, small LEDs	GPS, radios, board-to-board	Debugging C firmware, not Python

Choosing a Bus

Start from the device you are wiring up — it has usually already chosen for you.



The device only ever listens

An LED driver, a digital potentiometer, a DAC: **hardware SPI w/o MISO**.



You already spent the one SPI controller

Two SPI devices that cannot share a bus, or one with unusual timing. **SoftSPI** on any spare pins, at a tenth of the speed.



The datasheet is a table of registers

That is an **I2C** part. Two wires, an address to look up, `readfrom_mem` to fetch. Check whether the breakout carries its own pull-ups.



The device streams text at you

GPS receivers, LoRa and cellular modems, another microcontroller a metre away. **UART**, at whatever baud its datasheet names.



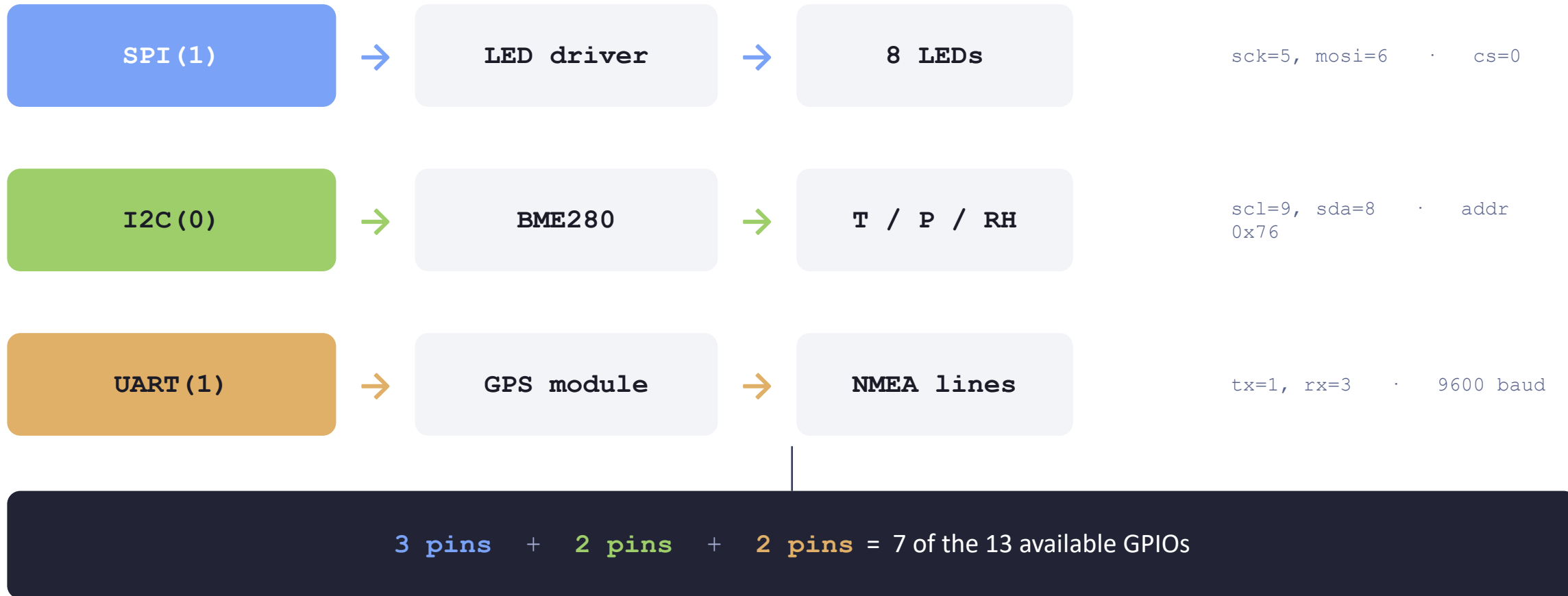
You have dropped out of MicroPython into C

Then **JTAG** stops being trivia: OpenOCD and gdb over the same USB-C cable, no probe to buy. Until then, `print()` is your debugger.

Four of the five are decided by the peripheral device. The only real choice is hardware SPI vs. SoftSPI,

All Three at Once

Nothing stops the buses running together — they are independent peripherals sharing only the CPU's attention.



3 protocols, 3 sets of timing, one 160 MHz core switching between them. The limit is pins, not throughput.